

Artwork Exception Resolution Agent

A six-day portfolio build for Sticker Mule's AI Agent Engineer role

Deliverable: Resolve an artwork blocker on a simulated sticker order: inspect the file, clarify customer intent, perform one eligible repair, verify it, and prepare a proof. Persist the case so work resumes after replies or failures. The three checks support this workflow. This document is a build plan; the application is not yet implemented.

Business fit and evidence

Sticker Mule's published guidance recommends raster artwork at 300 PPI or higher and CMYK colors. For artwork intended to bleed, it specifies 1/16 inch beyond the cut line [1]. Its feedback board includes requests for early resolution warnings and missing-font notifications [2]. These establish relevant workflows and customer interest, not measured internal costs or defect rates.

Honest positioning: This is an independent prototype based on public requirements. You have no access to Sticker Mule's production pipeline, customer files, printer profiles, or operational metrics. Do not claim to have reduced its reprints or replaced its proofing system.

Role fit: The verified vacancy emphasizes autonomous work and measured outcomes [3]. Its announced Grok rollout supports including Grok in a model comparison [4]. Neither source establishes artwork exceptions as its first internal priority.

The six-day scope

- **Inputs:** simulated order ID, customer request, PNG/JPEG artwork, rectangular sticker dimensions in inches or millimeters, border/full-bleed intent, and whether bleed is included. Confirm the trim rectangle; request clarification when placement is unknown.
- **Three checks:** effective resolution at print size; color mode/profile advisory; available bleed around the declared trim rectangle. Rules calculate measurements; the agent coordinates tools and explains results.
- **One real fix:** extend an eligible, opaque, uniform background into missing bleed. Preserve the original trim content exactly. Produce a new file and an annotated before/after preview.
- **Experience:** blocked order > inspect > clarify if needed > repair or escalate > verify > prepare proof. "Resolved" means only the supported artwork blocker is cleared. Customer proof approval and production release are separate, unperformed steps.

What stays outside v1

No expansion into Stores, Notify, Reply, or Ship. No arbitrary cut contours, SVG/PDF parsing, generative upscaling, automatic color conversion, print certification, payments, production release, factory integration, or multi-agent fleet. Stay with one artwork workflow.

Time and spending assumptions

Budget 40-48 focused hours for a developer familiar with Go/TypeScript. The order model affects UI, transitions, and tests; it is not a guaranteed 2-3-hour addition. Absorb it by limiting polish and integrations. Set a model-spend cap (for example \$20) and separate cloud alerts; these are budgets, not price estimates. Apply with verified work now and follow up after the build.

Checks and repair contract

Precise rules make the agent's decisions testable

1 / Resolution at the requested physical size

Formula: effective PPI = $\min(\text{trim pixels wide} / \text{finished inches wide}, \text{trim pixels high} / \text{finished inches high})$.

Convert millimeters using 25.4 mm per inch. Inspect the trim region, excluding bleed. A 600 × 600 pixel trim at 3 × 3 inches is 200 PPI; changing a metadata DPI tag does not add detail.

Flag values below the configured 300-PPI guideline [1]. Offer a smaller printable size or a higher-resolution source. Do not silently resize the order. Reject ambiguous stretching/cropping until the user confirms placement. Recalculate resolution whenever dimensions change.

2 / Color mode and embedded profile

Read the actual decoded mode and ICC profile presence. Report RGB, CMYK, grayscale, or unknown, and distinguish a missing profile from a known one. RGB produces an **advisory**: screen colors may differ in print. It is not a universal invalid-file condition. PNG does not support CMYK; JPEG can. Do not promise color accuracy or convert without a known destination profile.

3 / Bleed coverage

Bleed is artwork outside the intended cut that accommodates cutting variation. Run this check only for full-bleed intent and a confirmed trim rectangle. Measure coverage on all four sides in physical units; require 0.0625 inch per side for this configured sticker profile [1]. If trim/placement is unknown, return `NEEDS_INPUT` rather than guessing from edge pixels.

A 3-inch trim needs a 3.125-inch total extent. Convert required margins to pixels and round outward. At 300 PPI, 18.75 pixels becomes 19 pixels per side; report the resulting physical margin and unchanged trim size. Border designs do not automatically fail for missing bleed.

The repair: uniform-background extension

Eligibility: opaque image, confirmed rectangle, and an edge band that meets a documented uniform-color tolerance. Reject gradients, transparency, textured edges, or foreground objects touching the boundary. Passing eligibility is a conservative demo rule, not proof of manufacturing suitability.

Execution: generate a new canvas, fill new margins with the verified edge color, paste the original without resampling, and update trim coordinates. Save source and derived hashes. Re-run all checks. Require exact equality of decoded pixels inside the original content region; an encode/decode change must not hide under a perceptual score.

Exports: repaired lossless PNG, preview with trim/bleed overlays, and JSON diagnosis with dimensions and assumptions. Overlays belong only on the preview. If validation fails, discard the candidate and retain the original. CMYK inputs remain inspect-only in v1.

Result vocabulary

PASS / WARNING / NEEDS_INPUT / NEEDS_REVIEW per check; **REPAIRED** only after verification. “Pass” means the supported checks passed, not “certified print-ready.” Every result includes measured evidence, rule version, and the next action.

Order model and agent workflow

Preserve the stack; add a thin order layer around the artwork tools

Small, deployable architecture

TypeScript UI > Go GraphQL API > Postgres queue > Go worker > Python image tools. The worker uses one provider adapter for bounded decisions. Private storage holds artwork and proofs. UI polls case status. Python/Pillow owns only deterministic image operations; Go owns the workflow and validation.

GCP: Cloud Run services for Go/private Python; Cloud Run Job for queued work; Cloud SQL for Postgres; Cloud Storage for files; Secret Manager for credentials. Verify current deployment settings during implementation. Use Docker Compose locally.

Order, case, and execution states

orders: order_id, owner_id, product_type, declared width/height/unit, customer request, artwork_version, intent, trim rectangle, case_version. Link jobs, assets, findings, clarifications, repairs, and tool events through order_id. Log rule/model versions, hashes, timings, usage, and errors.

artwork_status: BLOCKED > AWAITING_CLARIFICATION > BLOCKED after reply > RESOLVED after verified checks; unsafe/unsupported cases become NEEDS_REVIEW. New artwork or dimensions reopen the case. These are prototype states, not Sticker Mule's internal schema.

Separate fields: proof_status = NOT_PREPARED or AWAITING_CUSTOMER_APPROVAL; production_status = NOT_RELEASED throughout v1. Execution jobs use QUEUED/RUNNING/SUCCEEDED/FAILED. A successful job can still leave the artwork case blocked. Warnings remain visible; all supported blockers must clear before RESOLVED.

Minimal API and repository

GraphQL: createOrder, createUpload(orderId), startResolution, answerClarification, requestRepair; order(id), job(id), findings, artifacts. Validate ownership, idempotency keys, and expected case_version on mutations. Reject stale replies; use signed upload URLs.

Layout: web/ · services/api-go/ · services/image-python/ · db/migrations/ · evals/ · infra/ · docs/. Keep a Docker Compose local environment, .env.example without secrets, and one documented start command.

A bounded autonomous loop

1. Load the order/version and claim the queued job lease.
2. Inspect artwork; compute checks; give structured evidence to the model.
3. Ask one targeted clarification, repair an eligible case, or escalate.
4. Persist a clarification and exit; a validated reply enqueues continuation.
5. Verify authorized repairs and recheck blockers. Prepare a proof, update the artwork state, and retain the complete audit trail.

Cap each execution segment at five tool calls and two transient retries. Persist the case budget across replies. One schema-repair attempt precedes deterministic fallback. The model cannot change measurements or override eligibility. Reclaim expired leases; repeated events must not duplicate repairs.

Security and reliability essentials

Enforce file signatures, 10 MB uploads, 25-megapixel decoded limits, sandboxed decoding, and timeouts. Use private files, short-lived links, owner checks, and seven-day deletion. Treat customer text/files as untrusted. Send metadata by default; redact logs and keep secrets server-side. Tie artifacts to order and artwork version.

Six-day execution plan

Ship a complete narrow workflow before adding features

Day 1 / Order model and working skeleton · 8 hours

Create the monorepo, containers, order/case/job migrations, GraphQL skeleton, case page, and private upload flow. Freeze rules and status semantics. **Done:** create blocked order > attach artwork > persist job > inspect through Go/Python. Show proof approval and production as separate states.

Day 2 / Deterministic checks · 8 hours

Implement image inspection, PPI, profile advisory, and confirmed-trim bleed measurement. Show findings on the order. Test units, orientation, and thresholds; freeze the scripted baseline. **Done:** correct findings and blockers on labeled cases, with RGB advisory handled separately from a blocking defect.

Day 3 / One verified repair · 8 hours

Implement conservative background eligibility, margin extension, immutable source storage, pixel equality verification, and preview overlays. Export diagnosis JSON and the repaired file. **Done:** one actual eligible defect is repaired; unsafe candidates are refused with reasons.

Day 4 / Agent loop and recovery · 8 hours

Connect one provider through a replaceable adapter. Implement clarification/reply/resume, case version checks, bounded tool calls, leases, and duplicate handling. **Done:** a case survives a reply and restart, verifies a repair, and clears its artwork blocker without approving the proof or releasing production.

Day 5 / Evaluation and deployment · 8 hours

Evaluate the agent against the fixed workflow; test state transitions, access controls, and deployment. If core gates pass and time remains, compare Grok and Claude on identical cases using page 5's protocol. **Done:** reproducible results and a deployed workflow; report provider comparison as deferred if unfinished.

Day 6 / Buffer, evidence, and application follow-up · 5 hours

Resolve remaining defects; record a three-minute demo; write the README, architecture, limitations, and case study. Verify a clean local setup from instructions. **Done:** repository, demo URL or recorded fallback, results, and a concise application follow-up are ready.

Cut order if time slips

Cut polish, extra integrations, and the optional second provider first. Keep the order workflow, clarification/resume, Go/GraphQL/Postgres path, three checks, verified repair, and baseline evaluation. If cloud setup blocks delivery, provide local containers and a recording, labeling GCP incomplete. Do not expand into other products or drop verification.

Scope gates and application timing

End of day 2: if deterministic checks are unreliable, spend day 3 fixing them before repair work. **End of day 4:** freeze features. **Release:** only supported passing cases may claim repair success. Unsupported cases must return a clear limitation. Apply using existing verified work now; send the finished evidence after six days. Do not present an unbuilt feature as complete.

Evaluation, demo, and evidence

Prove useful behavior and state exactly what remains unproven

A compact evaluation set

Create 60 fixtures: 30 development, 30 held-out; split original designs before variants. Include clean art, 299/300/301 PPI, profile variants, bleed, mixed issues, and unsafe repairs. Wrap held-out files in order cases with requests, scripted clarification replies, expected final states, and allowed actions. Test malformed files, stale replies, duplicate events, and restarts separately.

Release targets, not results: correct numeric checks; zero original-content pixel changes in accepted repairs; zero unsafe repairs or falsely resolved cases in the held-out set. No proof approval or production release. Report counts, precision/recall, repair coverage, and recovery failures. Synthetic tests cannot establish real-world reliability.

Show what the agent adds

Compare against rules plus scripted clarification/report templates. Measure correct blocker resolution, appropriate escalation, unnecessary questions, human intervention, latency, and cost per correctly resolved case. Grade final database state and tool effects, not persuasive wording. Keep deterministic code wherever the agent adds no measured benefit.

Optional Day 5 / Grok versus Claude

Use identical held-out orders, tools, replies, action limits, and grading. Freeze prompts; record model IDs, settings, tokens, latency, and cost. Reset state per run; repeat a subset if affordable. Compare completion and severe errors before cost. Publish counts and limitations, without claiming a general winner. Defer this comparison before compromising reliability.

Three-minute demonstration

0:00-0:30: blocked order, customer request, supported scope.

0:30-1:30: inspect > clarify > resume > eligible repair > verify.

1:30-2:15: clear artwork blocker; show proof awaiting approval. Demonstrate an unsafe case remaining in review.

2:15-3:00: restart recovery, audit trail, baseline results, and model comparison if completed.

Repository and hiring evidence checklist

Include setup, architecture/state diagram, API examples, migrations, fixture provenance, tests, frozen evaluation results, demo, and measured costs. Document actual GCP/provider coverage. Seek review of 5-10 examples from a print professional; otherwise disclose its absence. Savings remain hypothetical until observed with real users.

Application framing: "I built an independent agent that resolves a supported artwork blocker on simulated orders, asks for clarification, verifies a repair, and resumes after failures. Here are the demo and baseline results." Add the stack and model comparison only as implemented. Do not claim company savings or private integration.

Sources and limits · checked 15 September 2026

[1] [Official artwork requirements](#): resolution, conditional bleed, and CMYK recommendation; not output certification.

[2] [Public feedback board](#): resolution/font requests; no evidence of internal priorities or prevalence.

[3] [Verified AI Agent Engineer vacancy](#): same job ID; confirms stack and responsibilities. Closing date unknown.

[4] [August 2025 xAI announcement](#): planned Grok rollout to 370 employees and support use; not proof of completion or returns.